

06 - Relaties

📅 09-09-2026 ⌚ Ongeveer 31 minuten 📄 Ongeveer 4724 woorden

We hebben inmiddels een stevige basis opgebouwd met betrekking tot de kernprincipes van het ontwikkelen van een API.

In dit hoofdstuk gaan we onze kennis verder uitbreiden met het concept van relaties tussen tabellen, onderlinge relaties tussen modellen en introduceren het concept van relaties en `Navigation Properties`. We laten de simpele '**Laptops**' waar we tot nu toe mee gewerkt hebben achter ons en voegen nieuwe modellen en controllers toe om de basis van een kleine webshop te bouwen.

Terugblik & Focus

Wat voorafging: In de vorige lessen hebben we de fundamenten gelegd voor het ontwikkelen van een Web API. We hebben gezien hoe controllers werken, hoe we HTTP-requests afhandelen en hoe we eenvoudige CRUD-functionaliteit opzetten rondom een op zichzelf staand model (de Laptop).

De focus nu: In dit hoofdstuk verleggen we onze aandacht volledig naar de Data- en Modellaag van onze applicatie. We laten de eenvoudige, losstaande tabel achter ons en bouwen de database-architectuur voor een kleine webshop. We gaan ons specifiek toeleggen op het definiëren van onderlinge relaties (1:1, 1:N, N:M) tussen verschillende modellen met behulp van `Navigation Properties` en de `Fluent API`.

Modellen toevoegen.

Voeg aan de **Models** folder de volgende klassen toe of download de startbestanden van deze les, waar deze reeds aan zijn toegevoegd.

- **Bestelling.cs**

```
1 namespace WebAPIDemo.Models;
2
3 public class Bestelling
4 {
5     public int Id { get; set; }
```

csharp

```
6         public int KlantId { get; set; }
7
8         public Klant? Klant { get; set; } = default!;
9
10        public List<OrderLijn> OrderLijnen { get; set; } = default!;
11    }
12
```

- Klant.cs

```
1 namespace WebAPIDemo.Models;                                csharp
2
3 public class Klant
4 {
5     public int Id { get; set; }
6
7     public string Naam { get; set; } = default!;
8
9     public string Voornaam { get; set; } = default!;
10
11    public DateTime AangemaaktDatum { get; set; }
12
13    public List<Bestelling>? Bestellingen { get; set; } = default!;
14 }
```

- OrderLijn.cs

```
1 namespace WebAPIDemo.Models;                                csharp
2
3 public class OrderLijn
4 {
5     public int Id { get; set; }
6
7     public double Aantal { get; set; }
8
9     public int BestellingId { get; set; }
10
11    public int ProductId { get; set; }
12
13    public Product? Product { get; set; } = default!;
14
15    public Bestelling? Bestelling { get; set; } = default!;
16 }
```

- Product.cs

```
1 namespace WebAPIDemo.Models;
2
3 public class Product
4 {
5     public int Id { get; set; }
6
7     public string Naam { get; set; } = default!;
8
9     public string? Beschrijving { get; set; }
10
11     public decimal Prijs { get; set; }
12
13     public List<OrderLijn> OrderLijnen { get; set; } = default!;
14 }
```

Primary & Foreign Keys

Zoals je waarschijnlijk al weet uit eerdere databasevakken, bepalen **Primary Keys (PK's)** en **Foreign Keys (FK's)** de **relaties tussen tabellen**.

- **Primary Key:** De hoofdsleutel van een tabel. Deze is meestal een `int Id` en moet **uniek** zijn *binnen die tabel*. Als we zoeken op Klant 3, willen we uitsluitend de informatie van die specifieke klant zien.
- **Foreign Key:** Een verwijzing naar de Primary Key van een andere tabel. Zo kunnen we bijvoorbeeld zien dat een specifieke bestelling is geplaatst door Klant 3. Wie Klant 3 precies is, komen we te weten door in de tabel Klant te zoeken naar ID 3.

Dit principe werkt exact hetzelfde binnen onze C#-modellen. Dat is logisch, aangezien deze modellen de blauwdruk vormen voor onze database. Kijken we naar het model van een bestelling, dan zien we een eigenschap genaamd **KlantId**. Dit fungeert als de Foreign Key. Het vertelt ons (en de database) dat deze specifieke bestelling gekoppeld is aan exact één klant.

Modellen en Entiteiten

In de echte wereld worden modellen zelden direct gebruikt om databasetabellen te genereren of om mee te communiceren. Modellen zijn namelijk **volatiel**; *er worden vaak eigenschappen aan toegevoegd of uit verwijderd* voor specifiek gebruik in de API. Als modellen en tabellen 1-op-1 gelijk moeten blijven, zou er bij elke kleine aanpassing een nieuwe **database update** uitgevoerd moeten worden.

Dit kost veel tijd en zit experimentatie in de weg. Daarom gebruikt men in de professionele wereld **Entiteiten** (zie Entity Framework). Deze hebben exact dezelfde functionaliteit als onze modellen, maar waar een Model flexibel mag zijn, is een Entiteit *meer in steen gebeiteld*. Ze veranderen enkel wanneer de structuur van de database daadwerkelijk wijzigt. De data wordt vanuit de database ingeladen in de entiteiten en daarna omgezet naar modellen (of DTO's).

Om de complexiteit in deze cursus behapbaar te houden, laten we Entiteiten achterwege en gebruiken we onze Modellen voor beide doeleinden.

Relaties in Entity Framework

In de database wereld onderscheiden we drie soorten relaties :

Notitie

De code voor Persoon en Identiteitskaart is louter ter illustratie van de theorie en hoeft niet te worden overgeschreven in het project.

1. Eén-op-één relatie (1:1)

- Beide tabellen / modellen hebben een verwijzing (Primary/Foreign Key) naar elkaar.
- Beide modellen hebben een enkele Navigation Property naar elkaar.
- *Bijvoorbeeld:* Een persoon heeft één identiteitskaart. Een identiteitskaart kan maar toegewezen zijn aan één persoon.

```
1 public class Persoon
2 {
3     public int Id { get; set; }
4     public string Naam { get; set; } = default!;
5
6     // Navigation Property naar de unieke kaart
7     public Identiteitskaart? Identiteitskaart { get; set; }
```

csharp

```
7     }
8
9     public class Identiteitskaart
10    {
11        public int Id { get; set; }
12        public string KaartNummer { get; set; } = default!;
13
14        // Foreign Key naar Persoon
15        public int PersoonId { get; set; }
16
17        // Navigation Property terug naar de hoofdentiteit
18        public Persoon? Persoon { get; set; }
19    }
20
```

2. Eén-op-veel relatie (1:N)

- Eén tabel / model (de 'veel'-kant) heeft een Foreign Key naar de ander (de 'één'-kant).
- De 'veel'-kant heeft een enkele Navigation Property naar het hoofdmodel, terwijl het hoofdmodel een lijst (`List<T>`) van Navigation Properties heeft.
- *Bijvoorbeeld:* Een klant kan meerdere bestellingen plaatsen, maar één specifieke bestelling hoort altijd bij exact één klant.

```
1     public class Klant
2     {
3         public int Id { get; set; }
4         public string Naam { get; set; } = default!;
5
6         // De ene kant heeft een lijst van het andere model
7         public List<Bestelling>? Bestellingen { get; set; }
8     }
9
10    public class Bestelling
11    {
12        public int Id { get; set; }
13
14        // Foreign Key
15        public int KlantId { get; set; }
16
17        // De andere kant heeft een enkele Navigation Property
18        public Klant? Klant { get; set; }
19    }
```

csharp

Info

Merk op in het voorbeeld hierboven dat een één relatie zowel een Foreign key als een (enkele) Navigation Property heeft. Een Veel relatie heeft enkel een (lijst van) Navigation Property en geen Foreign Key.

Logisch, want een Foreign key wil zeggen: *maximaal één*.

3. Veel-op-veel relatie (N:M) Een doordenker: Als een Foreign Key betekent dat een record aan maximaal één ander record gekoppeld mag zijn, en 2 tabellen waarin 1 van beide een Foreign Key naar de ander heeft betekent dat ze een één op veel relatie hebben, hoe zorgen we er dan voor dat beide kanten een veel op veel relatie hebben met elkaar?

Hint: Het antwoord is niet in geen van beide Foreign keys plaatsen. In een database zonder relaties (FK's) zijn tabellen niet met elkaar verbonden en kunnen dan ook geen relaties hebben.

Hoe lossen we dit dan op? Door een tussentabel (associatietabel) tussen beide tabellen te plaatsen. Hierin leggen we **twee aparte één-op-veel relaties vast**. *Twee één-op-veel relaties vormen samen een veel-op-veel relatie*. In onze webshop is OrderLijn deze tussentabel.

```
1 public class Bestelling                                     csharp
2 {
3     public int Id { get; set; }
4
5     // Lijst naar de tussentabel
6     public List<OrderLijn> OrderLijnen { get; set; } = default!;
7 }
8
9 public class Product
10 {
11     public int Id { get; set; }
12
13     // Lijst naar de tussentabel
14     public List<OrderLijn> OrderLijnen { get; set; } = default!;
15 }
16
17 // De Tussentabel
18 public class OrderLijn
19 {
20     public int Id { get; set; }
21     public double Aantal { get; set; }
```

```
22
23     // FK en Navigation Property naar Bestelling (1 op N)
24     public int BestellingId { get; set; }
25     public Bestelling? Bestelling { get; set; }
26
27     // FK en Navigation Property naar Product (1 op N)
28     public int ProductId { get; set; }
29     public Product? Product { get; set; }
30 }
```

Relaties tussen de tabellen

Met deze kennis kunnen we de relaties tussen de modellen gaan ontleden:

Klant ↔ Bestelling (Eén-op-veel / 1:N)

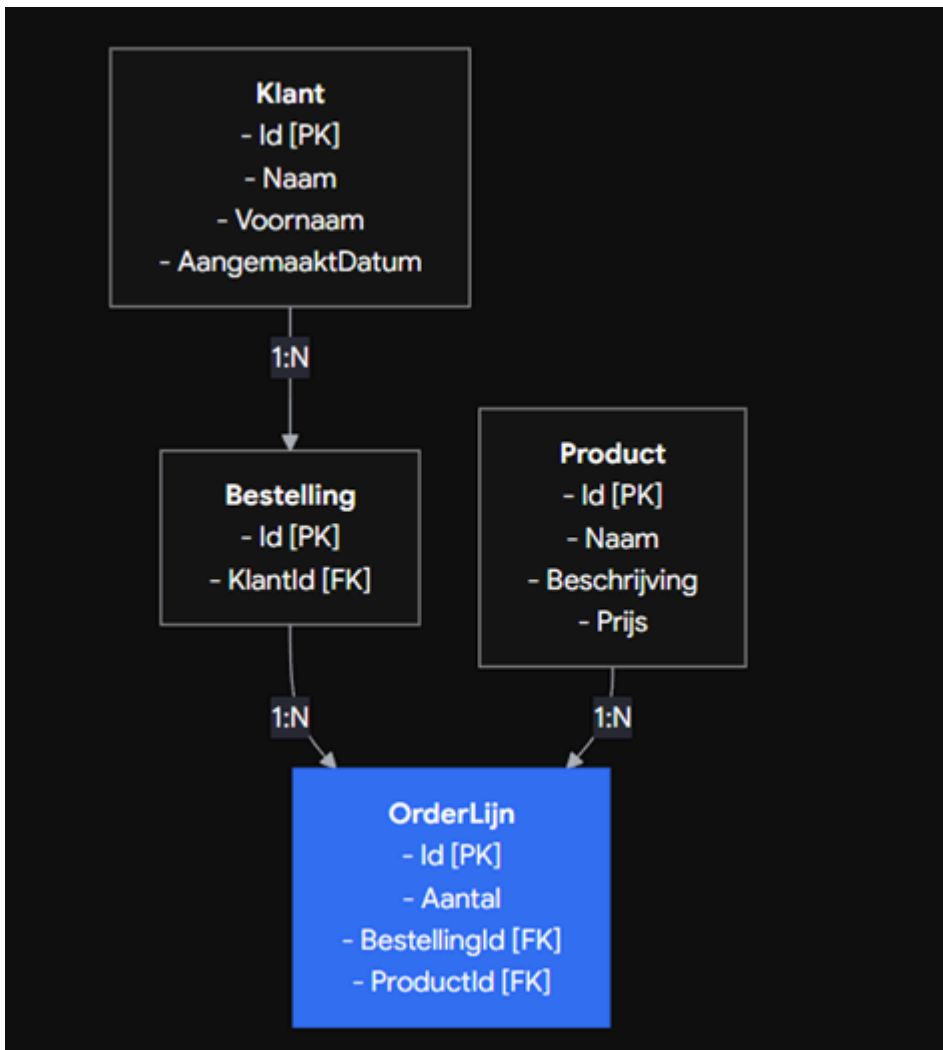
- Eén klant kan meerdere bestellingen plaatsen.
- Eén bestelling hoort bij exact één klant.
- Database: KlantId (Foreign Key) staat in de tabel Bestelling.

Bestelling ↔ Product (Veel-op-veel / N:M)

- Eén bestelling kan meerdere producten bevatten.
- Eén product kan in meerdere bestellingen voorkomen.
- Database: Kan niet direct gekoppeld worden, hiervoor is een tussentabel nodig.

OrderLijn (De Tussentabel)

- Knipt de veel-op-veel relatie op in twee overzichtelijke één-op-veel relaties.
- Database: Bevat BestellingId en ProductId als Foreign Keys om de link te leggen.
- Extra functie: Slaat specifieke informatie over de koppeling op, zoals het bestelde Aantal.



DbContext Bijwerken & Migratie Uitvoeren

Voeg je nieuwe **Modellen** toe als `DbSet<T>` properties aan je `WebAPIDemoContext`. Zodra je dit hebt gedaan, laat je een nieuwe migratie genereren, controleer je deze kort en update je de database.

```
1  using Microsoft.EntityFrameworkCore;
2  using WebAPIDemo.Models;
3
4  namespace WebApiDemo.Data
5  {
6      public class WebAPIDemoContext : DbContext
7      {
8          public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>
9              options)
10             : base(options) { }
```

```
11         public DbSet<Laptop> Laptops { get; set; }
12
13         public DbSet<Bestelling> Bestellingen { get; set; }
14
15         public DbSet<Klant> Klanten { get; set; }
16
17         public DbSet<OrderLijn> OrderLijnen { get; set; }
18
19         public DbSet<Product> Producten { get; set; }
20
21         protected override void OnModelCreating(ModelBuilder
22 modelBuilder)
23         {
24             base.OnModelCreating(modelBuilder);
25
26             modelBuilder.Entity<Laptop>(entity =>
27             {
28                 entity.ToTable("Laptops");
29                 entity.Property(p =>
30 p.Merk).IsRequired().HasMaxLength(50);
31                 entity.Property(p =>
32 p.Processor).IsRequired().HasMaxLength(100);
33                 entity.Property(p => p.RamInGB).IsRequired();
34                 entity.Property(p =>
35 p.Prijs).IsRequired().HasPrecision(18, 2);
36                 entity.Property(p =>
37 p.GPU).IsRequired().HasMaxLength(100);
38
39             });
40
41             modelBuilder.Entity<Bestelling>(entity =>
42             {
43                 entity.ToTable("Bestelling");
44
45
46             });
47
48             modelBuilder.Entity<Klant>(entity =>
49             {
50                 entity.ToTable("Klant");
51             });
52
53             modelBuilder.Entity<OrderLijn>(entity =>
```

```
54         {
55             entity.ToTable("OrderLijn");
56         });
57
58         modelBuilder.Entity<Product>(entity =>
59         {
60             entity.ToTable("Product");
61         });
62     }
63 }
```

```
1 Add-Migration add_tables
2 Update-Database
```

pwsh

Relaties leggen met Fluent API

Naast het definiëren van tabelnamen en kolom-eigenschappen (zoals `IsRequired()` of `HasMaxLength()`), is de `OnModelCreating` methode de aangewezen plek om relaties tussen je tabellen expliciet te definiëren. We doen dit met behulp van de Fluent API.

Door de relaties expliciet uit te schrijven, geef je Entity Framework (EF) exacte instructies over hoe de Foreign Keys (vreemde sleutels) in de database moeten worden aangemaakt.

Hoe lees je de Fluent API syntax? We lezen de configuratie altijd vanuit het perspectief van de entiteit die we op dat moment aan het configureren zijn.

Neem bijvoorbeeld de relatie tussen Bestelling en Klant:

- `HasOne(p => p.Klant)` : Een bestelling heeft één specifieke klant.
- `WithMany(x => x.Bestellingen)` : Die ene klant kan meerdere bestellingen hebben.
- `HasForeignKey(y => y.KlantId)` : De databasekolom die deze twee verbindt, is `KlantId`.
- `OnDelete(DeleteBehavior.Restrict)` : Dit is een belangrijke beveiliging! Standaard gebruikt EF Cascade Delete (verwijder je een klant, dan wist de database onherroepelijk al zijn bestellingen). Met `Restrict` blokkeer je het verwijderen van een klant zolang deze nog actieve bestellingen in het systeem heeft staan.

Waarschuwing

⚠️ Waarom `DeleteBehavior.Restrict` je baan kan redden. Standaard staat Entity Framework vaak ingesteld op **Cascade Delete**. Dit betekent: als je een hoofd-entiteit verwijdert, wist EF automatisch *alles* wat via een Foreign Key daaraan vastgekoppeld zit. Met een beetje pech trek je een ganse database leeg met 1 ongelukkig delete statement.

Stel je voor: Klant "Gerrit" belt de helpdesk. Gerrit is een beetje humeurig en wil absoluut uit het systeem verwijderd worden. Jij denkt als behulpzame developer: *Geen probleem Gerrit, we regelen het even voor je*. Je schrijft een snelle query en verwijdert de rij van Gerrit uit de tabel `Klant`.

Als je **Cascade Delete** aan had staan, gebeurt er achter de schermen dit: *Poef!* Niet alleen Gerrit is weg, maar de database verwijdert geruisloos ook al zijn 450 bestellingen van de afgelopen vijf jaar, inclusief alle gekoppelde orderlijnen en de complete omzetgeschiedenis die daarbij hoorde.

De volgende ochtend krijgt de financieel directeur een beroerte omdat de jaarcijfers plotseling met €80.000 zijn gedaald, de Belastingdienst staat voor de deur voor een audit, en jij mag op LinkedIn je status veranderen naar *"Beschikbaar voor een nieuwe uitdaging"*. 🤖

Met `DeleteBehavior.Restrict` gooit de database gelukkig de handrem erop. Als je Gerrit probeert te verwijderen, zegt SQL Server: *"Ho eens even, vriend. Ik ga Gerrit niet verwijderen, want er hangen nog bestellingen aan hem vast."* Restrict is de digitale veiligheidsgordel voor je bedrijfsdata. Nog een reden waarom soft deletes vaak boven hard deletes verkozen worden.



Vervang je bestaande WebAPIDemoContext door de onderstaande code om alle tabellen en relaties netjes te configureren:

```
1  using Microsoft.EntityFrameworkCore;
2  using WebAPIDemo.Models;
3
4  namespace WebApiDemo.Data
5  {
6      public class WebAPIDemoContext : DbContext
7      {
8          public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>
9  options)
10             : base(options) { }
11
12             public DbSet<Laptop> Laptops { get; set; }
13
14             public DbSet<Bestelling> Bestellingen { get; set; }
15
16             public DbSet<Klant> Klanten { get; set; }
17
18             public DbSet<OrderLijn> OrderLijnen { get; set; }
19
20             public DbSet<Product> Producten { get; set; }
21
22             protected override void OnModelCreating(ModelBuilder
23 modelBuilder)
24             {
25                 base.OnModelCreating(modelBuilder);
26
27                 modelBuilder.Entity<Laptop>(entity =>
28                 {
29                     entity.ToTable("Laptops");
30                     entity.Property(p =>
31 p.Merk).IsRequired().HasMaxLength(50);
32                     entity.Property(p =>
33 p.Processor).IsRequired().HasMaxLength(100);
34                     entity.Property(p => p.RamInGB).IsRequired();
35                     entity.Property(p =>
36 p.Prijs).IsRequired().HasPrecision(18, 2);
37                     entity.Property(p =>
38 p.GPU).IsRequired().HasMaxLength(100);
39
39                 });

```

```
40
41     modelBuilder.Entity<Bestelling>(entity =>
42     {
43         entity.ToTable("Bestelling");
44
45         entity.HasOne(p => p.Klant)
46             .WithMany(x => x.Bestellingen)
47             .HasForeignKey(y => y.KlantId)
48             .OnDelete(DeleteBehavior.Restrict)
49             .IsRequired();
50
51     });
52
53     modelBuilder.Entity<Klant>(entity =>
54     {
55         entity.ToTable("Klant");
56     });
57
58     modelBuilder.Entity<OrderLijn>(entity =>
59     {
60         entity.ToTable("OrderLijn");
61
62         entity.HasOne(p => p.Bestelling)
63             .WithMany(x => x.OrderLijnen)
64             .HasForeignKey(y => y.BestellingId)
65             .OnDelete(DeleteBehavior.Restrict)
66             .IsRequired();
67
68         entity.HasOne(p => p.Bestelling)
69             .WithMany(x => x.OrderLijnen)
70             .HasForeignKey(y => y.BestellingId)
71             .OnDelete(DeleteBehavior.Restrict)
72             .IsRequired();
73     });
74
75     modelBuilder.Entity<Product>(entity =>
76     {
77         entity.ToTable("Product");
78     });
79 }
80 }
```

```
1 Add-Migration add_relations
2 Update-Database
```

pwsh

Data toevoegen

Voer daarna onderstaand script uit in je database OF voeg de data seeding lijnen toe aan je **WebApiDbContext** **onModelCreating** override. In deze opleiding geven we er de voorkeur aan om te werken met Fluent API.

DataSeeding met Fluent API

```
1 using Microsoft.EntityFrameworkCore;
2 using WebAPIDemo.Models;
3
4 namespace WebApiDemo.Data
5 {
6     public class WebAPIDemoContext : DbContext
7     {
8         public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>
9 options)
10         : base(options) { }
11
12         public DbSet<Laptop> Laptops { get; set; }
13
14         public DbSet<Bestelling> Bestellingen { get; set; }
15
16         public DbSet<Klant> Klanten { get; set; }
17
18         public DbSet<OrderLijn> OrderLijnen { get; set; }
19
20         public DbSet<Product> Producten { get; set; }
21
22         protected override void OnModelCreating(ModelBuilder
23 modelBuilder)
24         {
25             base.OnModelCreating(modelBuilder);
26
27             modelBuilder.Entity<Laptop>(entity =>
28             {
29                 entity.ToTable("Laptops");
30                 entity.Property(p =>
```

csharp

```
30     p.Merk).IsRequired().HasMaxLength(50);
31         entity.Property(p =>
32     p.Processor).IsRequired().HasMaxLength(100);
33         entity.Property(p => p.RamInGB).IsRequired();
34         entity.Property(p =>
35     p.Prijs).IsRequired().HasPrecision(18, 2);
36         entity.Property(p =>
37     p.GPU).IsRequired().HasMaxLength(100);
38
39     });
40
41     modelBuilder.Entity<Bestelling>(entity =>
42     {
43         entity.ToTable("Bestelling");
44
45         entity.HasOne(p => p.Klant)
46             .WithMany(x => x.Bestellingen)
47             .HasForeignKey(y => y.KlantId)
48             .OnDelete(DeleteBehavior.Restrict)
49             .IsRequired();
50
51     });
52
53     modelBuilder.Entity<Klant>(entity =>
54     {
55         entity.ToTable("Klant");
56     });
57
58     modelBuilder.Entity<OrderLijn>(entity =>
59     {
60         entity.ToTable("OrderLijn");
61
62         entity.HasOne(p => p.Bestelling)
63             .WithMany(x => x.OrderLijnen)
64             .HasForeignKey(y => y.BestellingId)
65             .OnDelete(DeleteBehavior.Restrict)
66             .IsRequired();
67
68         entity.HasOne(p => p.Bestelling)
69             .WithMany(x => x.OrderLijnen)
70             .HasForeignKey(y => y.BestellingId)
71             .OnDelete(DeleteBehavior.Restrict)
72             .IsRequired();
```

```
73         });
74
75         modelBuilder.Entity<Product>(entity =>
76         {
77             entity.ToTable("Product");
78         });
79
80         // 1. Klanten Seeden
81         modelBuilder.Entity<Klant>().HasData(
82             new Klant { Id = 1, Naam = "Van Der Neffe", Voornaam =
83 "Leon", AangemaaktDatum = new DateTime(2022, 10, 1) },
84             new Klant { Id = 2, Naam = "Van De Kasseinen", Voornaam
85 = "Firmin", AangemaaktDatum = new DateTime(2022, 10, 2) },
86             new Klant { Id = 3, Naam = "Kiekeboe", Voornaam =
87 "Marcel", AangemaaktDatum = new DateTime(2022, 10, 3) }
88         );
89
90         // 2. Producten Seeden
91         // Let op de 'm' achter de prijzen om expliciet aan te
92 geven dat het om decimalen gaat.
93         modelBuilder.Entity<Product>().HasData(
94             new Product { Id = 1, Naam = "fiets", Prijs = 100.00m,
95 Beschrijving = "Dit is een fiets" },
96             new Product { Id = 2, Naam = "koersfiets", Prijs =
97 200.00m, Beschrijving = "Dit is een mooie koersfiets" },
98             new Product { Id = 3, Naam = "auto", Prijs = 2000.00m,
99 Beschrijving = "Dit is een auto" }
100        );
101
102        // 3. Bestellingen Seeden
103        // We geven enkel de Foreign Key (KlantId) mee, niet het
104 volledige Klant-object.
105        modelBuilder.Entity<Bestelling>().HasData(
106            new Bestelling { Id = 1, KlantId = 1 },
107            new Bestelling { Id = 2, KlantId = 2 },
108            new Bestelling { Id = 3, KlantId = 3 },
109            new Bestelling { Id = 4, KlantId = 2 },
110            new Bestelling { Id = 5, KlantId = 3 }
111        );
112
113        // 4. OrderLijnen Seeden
114        modelBuilder.Entity<OrderLijn>().HasData(
115            new OrderLijn { Id = 1, Aantal = 3, BestellingId = 1,
```

```
        ProductId = 1 },
        new OrderLijn { Id = 2, Aantal = 7, BestellingId = 1,
        ProductId = 2 },
        new OrderLijn { Id = 3, Aantal = 4, BestellingId = 2,
        ProductId = 1 },
        new OrderLijn { Id = 4, Aantal = 1, BestellingId = 2,
        ProductId = 2 },
        new OrderLijn { Id = 5, Aantal = 2, BestellingId = 3,
        ProductId = 1 },
        new OrderLijn { Id = 6, Aantal = 3, BestellingId = 4,
        ProductId = 1 },
        new OrderLijn { Id = 7, Aantal = 1, BestellingId = 4,
        ProductId = 3 },
        new OrderLijn { Id = 8, Aantal = 2, BestellingId = 5,
        ProductId = 1 },
        new OrderLijn { Id = 9, Aantal = 6, BestellingId = 5,
        ProductId = 2 },
        new OrderLijn { Id = 10, Aantal = 10, BestellingId = 5,
        ProductId = 3 }
    );
}
```

De code in onze OnModelCreating-methode wordt te lang en onoverzichtelijk. We gaan dit grote codeblok daarom refactoren en opsplitsen in twee methoden met elk één duidelijke verantwoordelijkheid (het Single Responsibility Principle).

De eerste methode bouwt de tabelstructuur op, de tweede methode voegt de initiële data toe (seeden).

Info

Vuistregel: een methode mag idealiter niet langer zijn dan één schermhoogte. Is dat wel het geval? Dan is het hoog tijd om te refactoren.

Tips

Slimmer werken met Visual Studio!

Selecteer de samenhangende code en gebruik de sneltoets Ctrl + R, M (of rechtermuisknop → Quick Actions and Refactorings → Extract Method). Visual Studio splitst de code dan automatisch op en herkent meestal direct de juiste in- en outputparameters.

```
1      using Microsoft.EntityFrameworkCore;
2      using WebAPIDemo.Models;
3
4      namespace WebApiDemo.Data
5      {
6          public class WebAPIDemoContext : DbContext
7          {
8              public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>
9              options)
10                 : base(options) { }
11
12              public DbSet<Laptop> Laptops { get; set; }
13
14              public DbSet<Bestelling> Bestellingen { get; set; }
15
16              public DbSet<Klant> Klanten { get; set; }
17
18              public DbSet<OrderLijn> OrderLijnen { get; set; }
19
20              public DbSet<Product> Producten { get; set; }
21
22              protected override void OnModelCreating(ModelBuilder
23              modelBuilder)
24              {
25                  base.OnModelCreating(modelBuilder);
26
27                  GenerateTables(modelBuilder);
28                  SeedData(modelBuilder);
29              }
30
31              private void GenerateTables(ModelBuilder modelBuilder)
32              {
33                  modelBuilder.Entity<Laptop>(entity =>
34                  {
```

```
35         entity.ToTable("Laptops");
36         entity.Property(p =>
37 p.Merk).IsRequired().HasMaxLength(50);
38         entity.Property(p =>
39 p.Processor).IsRequired().HasMaxLength(100);
40         entity.Property(p => p.RamInGB).IsRequired();
41         entity.Property(p =>
42 p.Prijs).IsRequired().HasPrecision(18, 2);
43         entity.Property(p =>
44 p.GPU).IsRequired().HasMaxLength(100);
45
46     });
47
48     modelBuilder.Entity<Bestelling>(entity =>
49     {
50         entity.ToTable("Bestelling");
51
52         entity.HasOne(p => p.Klant)
53             .WithMany(x => x.Bestellingen)
54             .HasForeignKey(y => y.KlantId)
55             .OnDelete(DeleteBehavior.Restrict)
56             .IsRequired();
57
58     });
59
60     modelBuilder.Entity<Klant>(entity =>
61     {
62         entity.ToTable("Klant");
63     });
64
65     modelBuilder.Entity<OrderLijn>(entity =>
66     {
67         entity.ToTable("OrderLijn");
68
69         entity.HasOne(p => p.Bestelling)
70             .WithMany(x => x.OrderLijnen)
71             .HasForeignKey(y => y.BestellingId)
72             .OnDelete(DeleteBehavior.Restrict)
73             .IsRequired();
74
75         entity.HasOne(p => p.Bestelling)
76             .WithMany(x => x.OrderLijnen)
77             .HasForeignKey(y => y.BestellingId)
```

```
78         .OnDelete(DeleteBehavior.Restrict)
79         .IsRequired();
80     });
81
82     modelBuilder.Entity<Product>(entity =>
83     {
84         entity.ToTable("Product");
85     });
86 }
87
88 private void SeedData(ModelBuilder modelBuilder)
89 {
90     // 1. Klanten Seeden
91     modelBuilder.Entity<Klant>().HasData(
92         new Klant { Id = 1, Naam = "Van Der Neffe", Voornaam =
93 "Leon", AangemaaktDatum = new DateTime(2022, 10, 1) },
94         new Klant { Id = 2, Naam = "Van De Kasseinen", Voornaam =
95 "Firmin", AangemaaktDatum = new DateTime(2022, 10, 2) },
96         new Klant { Id = 3, Naam = "Kiekeboe", Voornaam =
97 "Marcel", AangemaaktDatum = new DateTime(2022, 10, 3) }
98     );
99
100     // 2. Producten Seeden
101     // Let op de 'm' achter de prijzen om expliciet aan te geven
102     dat het om decimalen gaat.
103     modelBuilder.Entity<Product>().HasData(
104         new Product { Id = 1, Naam = "fiets", Prijs = 100.00m,
105 Beschrijving = "Dit is een fiets" },
106         new Product { Id = 2, Naam = "koersfiets", Prijs =
107 200.00m, Beschrijving = "Dit is een mooie koersfiets" },
108         new Product { Id = 3, Naam = "auto", Prijs = 2000.00m,
109 Beschrijving = "Dit is een auto" }
110     );
111
112     // 3. Bestellingen Seeden
113     // We geven enkel de Foreign Key (KlantId) mee, niet het
114     volledige Klant-object.
115     modelBuilder.Entity<Bestelling>().HasData(
116         new Bestelling { Id = 1, KlantId = 1 },
117         new Bestelling { Id = 2, KlantId = 2 },
118         new Bestelling { Id = 3, KlantId = 3 },
119         new Bestelling { Id = 4, KlantId = 2 },
120         new Bestelling { Id = 5, KlantId = 3 }
```

```

121         );
122
123         // 4. OrderLijnen Seeden
124         modelBuilder.Entity<OrderLijn>().HasData(
            new OrderLijn { Id = 1, Aantal = 3, BestellingId = 1,
            ProductId = 1 },
            new OrderLijn { Id = 2, Aantal = 7, BestellingId = 1,
            ProductId = 2 },
            new OrderLijn { Id = 3, Aantal = 4, BestellingId = 2,
            ProductId = 1 },
            new OrderLijn { Id = 4, Aantal = 1, BestellingId = 2,
            ProductId = 2 },
            new OrderLijn { Id = 5, Aantal = 2, BestellingId = 3,
            ProductId = 1 },
            new OrderLijn { Id = 6, Aantal = 3, BestellingId = 4,
            ProductId = 1 },
            new OrderLijn { Id = 7, Aantal = 1, BestellingId = 4,
            ProductId = 3 },
            new OrderLijn { Id = 8, Aantal = 2, BestellingId = 5,
            ProductId = 1 },
            new OrderLijn { Id = 9, Aantal = 6, BestellingId = 5,
            ProductId = 2 },
            new OrderLijn { Id = 10, Aantal = 10, BestellingId = 5,
            ProductId = 3 }
        );
    }
}

```

```

1 add-migration seed_data
2 update-database

```

bash

@tab PostGres

```

1
2 -- =====
3 -- 1. KLANTEN INVOEGEN
4 -- =====
5 INSERT INTO "Klant" ("Id", "Naam", "Voornaam", "AangemaaktDatum") VALUES
6 (1, 'Van Der Neffe', 'Leon', '2022-10-01 00:00:00'),
7 (2, 'Van De Kasseinen', 'Firmin', '2022-10-02 00:00:00'),
8 (3, 'Kiekeboe', 'Marcel', '2022-10-03 00:00:00');

```

postgres

```
9
10 -- Sequence synchroniseren voor de tabel Klant
11 SELECT setval(pg_get_serial_sequence('"Klant"', 'Id'),
12 COALESCE(MAX("Id"), 1)) FROM "Klant";
13
14
15 -- =====
16 -- 2. PRODUCTEN INVOEGEN
17 -- =====
18 INSERT INTO "Product" ("Id", "Naam", "Prijs", "Beschrijving") VALUES
19 (1, 'fiets', 100.00, 'Dit is een fiets'),
20 (2, 'koersfiets', 200.00, 'Dit is een mooie koersfiets'),
21 (3, 'auto', 2000.00, 'Dit is een auto');
22
23 -- Sequence synchroniseren voor de tabel Product
24 SELECT setval(pg_get_serial_sequence('"Product"', 'Id'),
25 COALESCE(MAX("Id"), 1)) FROM "Product";
26
27
28 -- =====
29 -- 3. BESTELLINGEN INVOEGEN
30 -- =====
31 INSERT INTO "Bestelling" ("Id", "KlantId") VALUES
32 (1, 1),
33 (2, 2),
34 (3, 3),
35 (4, 2),
36 (5, 3);
37
38 -- Sequence synchroniseren voor de tabel Bestelling
39 SELECT setval(pg_get_serial_sequence('"Bestelling"', 'Id'),
40 COALESCE(MAX("Id"), 1)) FROM "Bestelling";
41
42
43 -- =====
44 -- 4. ORDERLIJNEN INVOEGEN
45 -- =====
46 INSERT INTO "OrderLijn" ("Id", "Aantal", "BestellingId", "ProductId")
47 VALUES
48 (1, 3, 1, 1),
49 (2, 7, 1, 2),
50 (3, 4, 2, 1),
51 (4, 1, 2, 2),
```

```

52      (5, 2, 3, 1),
53      (6, 3, 4, 1),
54      (7, 1, 4, 3),
55      (8, 2, 5, 1),
56      (9, 6, 5, 2),
      (10, 10, 5, 3);

-- Sequence synchroniseren voor de tabel OrderLijn
SELECT setval(pg_get_serial_sequence('"OrderLijn"', 'Id'),
COALESCE(MAX("Id"), 1)) FROM "OrderLijn";

```

@tab Sql Server

```

1      SET IDENTITY_INSERT [dbo].[Klant] ON
2      INSERT INTO [dbo].[Klant] ([Id], [Naam], [Voornaam], [AangemaaktDatum])
3      VALUES (1, N'Van Der Neffe', N'Leon', N'2022-10-01 00:00:00')
4      INSERT INTO [dbo].[Klant] ([Id], [Naam], [Voornaam], [AangemaaktDatum])
5      VALUES (2, N'Van De Kasseinen', N'Firmin', N'2022-10-02 00:00:00')
6      INSERT INTO [dbo].[Klant] ([Id], [Naam], [Voornaam], [AangemaaktDatum])
7      VALUES (3, N'Kiekeboe', N'Marcel', N'2022-10-03 00:00:00')
8      SET IDENTITY_INSERT [dbo].[Klant] OFF
9
10     SET IDENTITY_INSERT [dbo].[Product] ON
11     INSERT INTO [dbo].[Product] ([Id], [Naam], [Prijs], [Beschrijving])
12     VALUES (1, N'fiets', CAST(100.00 AS Decimal(18, 2)), N'Dit is een fiets')
13     INSERT INTO [dbo].[Product] ([Id], [Naam], [Prijs], [Beschrijving])
14     VALUES (2, N'koersfiets', CAST(200.00 AS Decimal(18, 2)), N'Dit is een
15     mooie koersfiets')
16     INSERT INTO [dbo].[Product] ([Id], [Naam], [Prijs], [Beschrijving])
17     VALUES (3, N'auto', CAST(2000.00 AS Decimal(18, 2)), N'Dit is een auto')
18     SET IDENTITY_INSERT [dbo].[Product] OFF
19
20     SET IDENTITY_INSERT [dbo].[Bestelling] ON
21     INSERT INTO [dbo].[Bestelling] ([Id], [KlantId]) VALUES (1, 1)
22     INSERT INTO [dbo].[Bestelling] ([Id], [KlantId]) VALUES (2, 2)
23     INSERT INTO [dbo].[Bestelling] ([Id], [KlantId]) VALUES (3, 3)
24     INSERT INTO [dbo].[Bestelling] ([Id], [KlantId]) VALUES (4, 2)
25     INSERT INTO [dbo].[Bestelling] ([Id], [KlantId]) VALUES (5, 3)
26     SET IDENTITY_INSERT [dbo].[Bestelling] OFF
27
28     SET IDENTITY_INSERT [dbo].[OrderLijn] ON
29     INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],
[ProductId]) VALUES (1, 3, 1, 1)

```

```
30  INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
31  [ProductId]) VALUES (2, 7, 1, 2)  
32  INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (3, 4, 2, 1)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (4, 1, 2, 2)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (5, 2, 3, 1)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (6, 3, 4, 1)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (7, 1, 4, 3)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (8, 2, 5, 1)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (9, 6, 5, 2)  
    INSERT INTO [dbo].[OrderLijn] ([Id], [Aantal], [BestellingId],  
    [ProductId]) VALUES (10, 10, 5, 3)  
    SET IDENTITY_INSERT [dbo].[OrderLijn] OFF
```

...

Tips

Met data seeding via de Fluent API wordt data automatisch toegevoegd zodra de database wordt aangemaakt. Handmatige scripts zijn hierdoor niet meer nodig.

Repositories aanmaken

Nu we nieuwe **modellen** hebben aangemaakt, moeten er ook nieuwe **Repositories** gebouwd worden om deze data te kunnen ophalen en wegschrijven. In een latere stap koppelen we hier uiteraard ook weer **Controllers** aan. Voorlopig focussen we ons uitsluitend op het model **Bestelling**. De overige modellen behandelen we later.

We herhalen het beproefde recept: we maken eerst een interface aan (het contract) en implementeren deze daarna in een concrete repository-klasse.

1. De Interface: IBestellingRepository.cs

Maak in de map **Repositories** een nieuwe interface aan genaamd **IBestellingRepo.cs** en voeg de volgende CRUD-definities toe:

```
1  using WebAPIDemo.Models;                                csharp
2
3  namespace WebApiDemo.Repositories
4  {
5      public interface IBestellingRepository
6      {
7          Task<Bestelling> CreateAsync(Bestelling bestelling);
8          Task DeleteAsync(int id);
9          Task<List<Bestelling>> GetAllAsync();
10         Task<Bestelling?> GetByIdAsync(int id);
11         Task UpdateAsync(int id, Bestelling updatedBestelling);
12     }
13 }
```

2. De Klasse: BestellingRepository.cs

Maak in dezelfde map een klasse aan genaamd **BestellingRepository.cs** die deze interface implementeert en vul de methodes aan:

```
1  using Microsoft.EntityFrameworkCore;                      csharp
2  using WebApiDemo.Data;
3  using WebAPIDemo.Models;
4
5  namespace WebApiDemo.Repositories
6  {
7      public class BestellingRepository : IBestellingRepository
8      {
9          private readonly WebAPIDemoContext _context;
10
11         public BestellingRepository(WebAPIDemoContext context)
12         {
13             _context = context;
14         }
15     }
```

```
15     public async Task<Bestelling> CreateAsync(Bestelling bestelling)
16     {
17         _context.Bestellingen.Add(bestelling);
18         await _context.SaveChangesAsync();
19
20         return bestelling;
21     }
22
23     public async Task<List<Bestelling>> GetAllAsync()
24     {
25         return await _context.Bestellingen.ToListAsync();
26     }
27
28     public async Task<Bestelling?> GetByIdAsync(int id)
29     {
30         return await _context.Bestellingen.FindAsync(id);
31     }
32
33     // U - Update: Pas de gegevens aan
34     public async Task UpdateAsync(int id, Bestelling
35 updatedBestelling)
36     {
37         Bestelling? existingBestelling = await
38 _context.Bestellingen.FindAsync(id);
39
40         if (existingBestelling != null)
41         {
42             existingBestelling = updatedBestelling;
43
44             await _context.SaveChangesAsync();
45         }
46     }
47
48     // D - Delete: Verwijder een laptop
49     public async Task DeleteAsync(int id)
50     {
51         Bestelling? bestellingToDelete = await
52 _context.Bestellingen.FindAsync(id);
53
54         if (bestellingToDelete != null)
55         {
56             _context.Bestellingen.Remove(bestellingToDelete);
57         }
```

```
58         await _context.SaveChangesAsync();
59     }
    }
}
```

Info

Merk op dat de code in deze Interface en Repo bijna 100% hetzelfde is als de code in LaptopRepo. Repositories zijn zeer gelijkaardig. Meestal doen ze exact hetzelfde (1 object ophalen, alle objecten ophalen, een object updaten, ...) en is het enige wat verschilt de tabel of klasse waarmee ze interacteren. Daarnaast zijn er vaak maar enkele specifieke extra methodes, zoals het ophalen van de tien best verkopende producten.

Dit leidt uiteraard tot zeer veel herhalende code (*boilerplate code*), wat we als developers proberen te vermijden. (Onthoud: Een goede programmeur is lui.) In het volgende hoofdstuk bespreken we `Generics<T>` om al deze repetitieve repository-taken met één gedeelde codebase op te vangen.

Tips

Geen zin om eerst de Interface te typen en dan pas de klasse? Je bent zeker niet de enige! Vaak wordt een interface pas achteraf gegenereerd op basis van een bestaande klasse.

Laat Visual Studio het zware werk voor je doen:

1. Schrijf eerst je volledige klasse (BestellingRepo.cs) met alle publieke methoden.
2. Gebruik de sneltoets Ctrl + R, Ctrl + I (of rechtermuisklik op de klassenaam → Refactor → Extract Interface).
3. Visual Studio genereert nu automatisch een interface op basis van de **publieke methoden en properties**.

Repositories registreren in Program.cs

Voeg de volgende regel toe aan je **Program.cs** boven builder., en vergeet niet je using statements aan te vullen indien nodig:

```
1 // Registreer de BestellingRepository c#
2 builder.Services.AddScoped<IBestellingRepository, BestellingRepository>
   ();
```

Info

 Scoped? Door `AddScoped` te gebruiken, zorgen we ervoor dat er per HTTP-request één exemplaar van de repository wordt aangemaakt. Alle database-acties binnen dat ene verzoek delen zo dezelfde database-context.

Controllers aanmaken

Nu de data-laag (Repository & Model) volledig is opgezet en geregistreerd, kunnen we de buitenste laag van onze API bouwen: de Controller. Deze fungeert als de digitale ober die de HTTP-verzoeken van de klant (bijv. een frontend-applicatie of Swagger) aanneemt, doorgeeft aan de repository, en het resultaat netjes serveert.

Maak in de map **Controllers** een nieuw bestand aan genaamd `BestellingenController.cs` en voeg de volgende code toe:

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebAPIDemo.Models;
3  using WebApiDemo.Repositories;
4
5  namespace WebAPIDemo.Controllers
6  {
7      [Route("api/[controller]")]
8      [ApiController]
9      public class BestellingController : ControllerBase
10     {
11         private readonly IBestellingRepository _bestellingRepo;
12
13         // Dependency Injection
14         public BestellingController(IBestellingRepository bestellingRepo)
15         {
16             _bestellingRepo = bestellingRepo;
17         }
18
19         // GET: api/bestellingen
20         [HttpGet]
21         public async Task<ActionResult<IEnumerable<Bestelling>>>
22         GetBestellingen()
23         {
24             var bestellingen = await _bestellingRepo.GetAllAsync();
25             return Ok(bestellingen);
26         }
27     }
28 }
```

```
26     }
27
28     // GET: api/bestellingen/5
29     [HttpGet("{id}")]
30     public async Task<ActionResult<Bestelling>> GetBestelling(int id)
31     {
32         var bestelling = await _bestellingRepo.GetByIdAsync(id);
33
34         if (bestelling == null)
35         {
36             return NotFound($"Bestelling met ID {id} is niet
37 gevonden.");
38         }
39
40         return Ok(bestelling);
41     }
42
43     // POST: api/bestellingen
44     [HttpPost]
45     public async Task<ActionResult<Bestelling>>
46 PostBestelling(Bestelling bestelling)
47     {
48         var nieuweBestelling = await
49 _bestellingRepo.CreateAsync(bestelling);
50
51         return CreatedAtAction(nameof(GetBestelling), new { id =
52 nieuweBestelling.Id }, nieuweBestelling);
53     }
54
55     // PUT: api/bestellingen/5
56     [HttpPut("{id}")]
57     public async Task<IActionResult> PutBestelling(int id, Bestelling
58 bestelling)
59     {
60         if (id != bestelling.Id)
61         {
62             return BadRequest("De ID in de URL komt niet overeen met
63 het ID in de data.");
64         }
65
66         await _bestellingRepo.UpdateAsync(id, bestelling);
67         return NoContent();
68     }
```

```
69
70 // DELETE: api/bestellingen/5
71 [HttpDelete("{id}")]
72 public async Task<IActionResult> DeleteBestelling(int id)
73 {
74     var bestelling = await _bestellingRepo.GetByIdAsync(id);
75     if (bestelling == null)
76     {
77         return NotFound();
78     }
79
80     await _bestellingRepo.DeleteAsync(id);
81     return NoContent();
82 }
83 }
```

Start je applicatie op. Als je nu naar je Scalar interface navigeert, zie je een gloednieuw Bestellingen endpoint waarmee je de data in je SQL-database volledig kunt beheren!

Samenvatting

- Navigation Properties worden gebruikt om:
 - Relaties tussen modellen in C# te leggen.
 - Data uit meerdere tabellen gelijktijdig te kunnen ophalen.
- Primary & Foreign Keys
 - Bepalen de relaties tussen tabellen.
 - Een Foreign Key (zoals KlantId) verwijst altijd naar de Primary Key (Id) van een ander model.
- Relaties configureren via OnModelCreating() (Fluent API)
 - **HasOne()**: Definieert de 'één'-kant van een relatie.
 - **WithMany()**: Definieert de 'veel'-kant van een relatie.
 - **HasForeignKey()**: Definieert expliciet welke property de vreemde sleutel in de database is.

- **OnDelete(DeleteBehavior.Restrict)**: Schakelt de standaard Cascade Delete uit om je data te beveiligen tegen onbedoeld gegevensverlies.
- Veel-op-veel (N:M) relaties
 - Deze kunnen niet direct in een relationele database worden gelegd.
 - Oplossing: Gebruik een tussentabel (associatietabel, zoals OrderLijn). Hiermee splits je de N:M-relatie op in twee beheersbare 1:N-relaties.
- Data Seeding
 - Het toevoegen van voorbeelddata zodat we niet met een lege database hoeven te werken.
 - Dit kan door manueel een script uit te voeren of Seeding logica met Fluent API te voorzien je DbContext.
 - In deze oplossing verkiezen we FluentAPI.

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM